

MPS/MPO 张量网络方法详解

以 N 皇后问题为例的完整教程

张量网络学习笔记

2026 年 3 月

目录

1 从零开始：什么是张量？	2
1.1 标量、向量、矩阵、张量	2
1.2 张量的图形表示	2
2 张量缩并：最核心的运算	3
2.1 什么是缩并	3
2.2 图形上的缩并：连线 = 求和	4
3 N 皇后问题：从棋盘到张量网络	4
3.1 N 皇后问题回顾	4
3.2 关键思想：把约束变成张量	4
3.3 核心张量 B 的 9 个指标	5
3.4 B 张量非零元素的完整解读	5
3.4.1 情况一：格子为空 ($p = 1$)，无皇后经过	5
3.4.2 情况二：格子为空 ($p = 1$)，有信号”透传”	6
3.4.3 情况三：格子有皇后 ($p = 2$)	7
3.4.4 为什么只有 16 个非零元素	7
3.5 一个极简例子： 2×2 的情况	8
4 从 9 阶张量到 2D 网络张量：辅助张量的作用	8
4.1 问题：对角线指标如何处理？	8
4.2 解决方案概述：用 UE、DE、V0 把对角线变成上下左右	8
4.3 UE 张量详解：对角线信号的编码器	9
4.3.1 第一步：4 维 SWAP 张量	9
4.3.2 第二步：Reshape 为 $[2, 2, 4]$	9
4.3.3 UE 的作用方式	10

4.4	DE 张量详解：对角线信号的解码器	11
4.4.1	从单位矩阵出发	11
4.4.2	DE 的作用方式	12
4.5	V0 张量：对占据状态求和	13
4.6	UE 与 DE 的协作：对角线信号的完整路由	13
4.7	Reshape 后的键维度解读	14
5	完整数值推导：$T_{1,1}$ 的每一个元素	15
5.1	第一步：原始 9 阶张量（2 个非零元素）	15
5.2	第二步：与 UE 缩并（ dr 信号编码）	15
5.2.1	逐项计算	16
5.3	第三步：Reshape 为 $[8, 4]$ 矩阵	17
5.4	$T_{1,1}$ 的完整 8×4 矩阵	17
5.5	行指标和列指标的完整含义	18
5.5.1	行指标 I （垂直键，维度 8）= 向下传递的信息	18
5.5.2	列指标 J （水平键，维度 4）= 向右传递的信息	19
5.6	4 个非零元素的物理解读	19
5.7	验证：用 MATLAB 计算	20
6	MPS：矩阵乘积态	21
6.1	什么是 MPS	21
6.2	第 1 行作为 uMPS	22
6.3	第 4 行作为 dMPS	22
7	MPO：矩阵乘积算符	22
7.1	什么是 MPO	22
7.2	第 2、3 行作为 MPO	23
8	核心操作：MPO 作用于 MPS	23
8.1	概览	24
8.2	逐格点详解：uMPS 吸收第 2 行 MPO	24
8.2.1	格点 $j = 1$ （左端点）	24
8.2.2	格点 $j = 2$ （中间点）	25
8.2.3	格点 $j = 3$ （同 $j = 2$ ）	25
8.2.4	格点 $j = 4$ （右端点）	26
8.3	吸收后的 uMPS 全貌	26
9	下方 MPS 吸收 MPO（反方向）	26
9.1	dMPS 吸收第 3 行	26

10 最终缩并：两个 MPS 的内积	27
10.1 全局图景	27
10.2 拉链式缩并	27
10.2.1 第 1 列	28
10.2.2 第 2 列	28
10.2.3 第 3 列：同第 2 列	28
10.2.4 第 4 列（最后一列）	28
11 完整流程总结	29
11.1 计算复杂度	29
11.2 如何拓展到更大的 N?	30
12 附录：所有张量的详细形状一览	30
12.1 4×4 所有格点张量	30
12.2 缩并过程中的 MPS 形状变化	31
13 直觉总结	31

1 从零开始：什么是张量？

1.1 标量、向量、矩阵、张量

核心概念：张量的阶数

张量就是多维数组，它的阶数（也叫秩）等于它有多少个指标（下标）。

- 0 阶张量 = 标量（一个数）： $c = 3.14$
- 1 阶张量 = 向量（一系列数）： v_i ，如 $v = (1, 0, 1)$
- 2 阶张量 = 矩阵（一张表）： M_{ij} ，如 $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
- 3 阶张量： T_{ijk} ，一个“立方体”的数
- n 阶张量： $T_{i_1 i_2 \dots i_n}$ ， n 个指标

例子：3 阶张量

一个形状为 $[2, 3, 2]$ 的 3 阶张量 T_{ijk} ：

- 第 1 个指标 i 取值 $\{1, 2\}$ （维度 2）
- 第 2 个指标 j 取值 $\{1, 2, 3\}$ （维度 3）
- 第 3 个指标 k 取值 $\{1, 2\}$ （维度 2）
- 总共有 $2 \times 3 \times 2 = 12$ 个元素

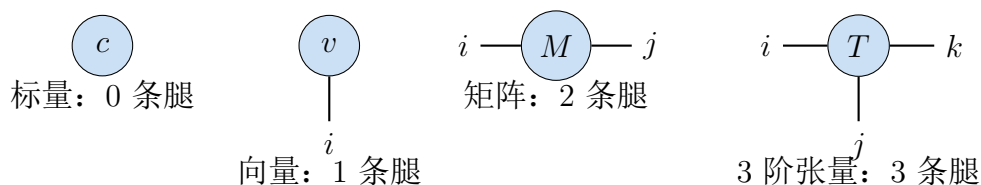
你可以把它想象成 2 张 3×2 的表格叠在一起。

1.2 张量的图形表示

在张量网络领域，我们用图形来表示张量，规则非常简单：

图形规则

- 一个张量 = 一个节点（圆圈或方块）
- 每个指标 = 从节点伸出的一条线（称为“腿”，leg）
- 线的维度 = 这个指标可以取多少个值



2 张量缩并：最核心的运算

2.1 什么是缩并

张量缩并 = 广义的矩阵乘法

两个张量缩并 (contraction)，就是把它们共享的指标求和。

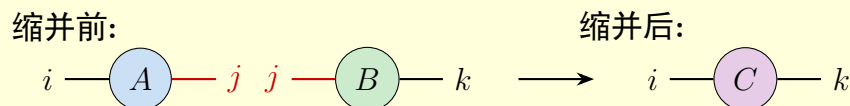
矩阵乘法 $C_{ik} = \sum_j A_{ij} B_{jk}$ 就是最简单的缩并：对共享指标 j 求和。

例子 1：矩阵乘法就是缩并

设 A 是 2×3 矩阵， B 是 3×4 矩阵：

$$C_{ik} = \sum_{j=1}^3 A_{ij} B_{jk}$$

图形表示：把 A 的右腿和 B 的左腿连起来。



红色的 j 指标被求和掉了，只剩下 i 和 k 。结果 C 是 2×4 矩阵。

例子 2：向量内积

$$s = \sum_{i=1}^n u_i v_i = \mathbf{u} \cdot \mathbf{v}$$

两个向量各有 1 条腿，缩并后 0 条腿 $\rightarrow$ 标量。



例子 3：高阶张量缩并

设 A_{ijkl} (4 阶) 和 B_{kmp} (3 阶)，对 k 缩并：

$$C_{ijlmp} = \sum_k A_{ijkl} B_{kmp}$$

结果是 5 阶张量 ($4 + 3 - 2 = 5$)。

规则：每缩并一对指标，总腿数减 2。

2.2 图形上的缩并：连线 = 求和

核心规则

在张量网络图中：

- 悬空的腿（自由指标）= 结果张量的指标
- 两个节点之间的连线（内部指标）= 被求和的指标
- 连线的维度 = 求和的范围

3 N 皇后问题：从棋盘到张量网络

3.1 N 皇后问题回顾

在 $N \times N$ 的棋盘上放 N 个皇后，使得：

- 每行恰好 1 个皇后
- 每列恰好 1 个皇后
- 每条对角线最多 1 个皇后

问：有多少种合法的放法？

3.2 关键思想：把约束变成张量

核心映射

1. 棋盘的每个格子 (i, j) 放一个张量 $T_{i,j}$
2. 张量的指标沿 8 个方向（上、下、左、右、四个对角）传递”攻击信号”
3. 每个格子有一个状态 p ： $p = 1$ （空）或 $p = 2$ （有皇后）
4. 张量的非零元素编码了 **局部约束**：如果这里有皇后，就必须向所有方向发出攻击信号
5. 将所有张量缩并起来（对所有内部指标求和），得到的标量就是合法配置的总数

这和统计力学中的配分函数完全类似：

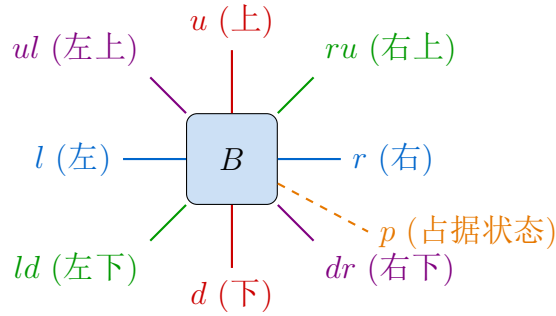
$$Z_N = \sum_{\text{所有配置}} \prod_{\text{所有格点}} W_{\text{格点}}$$

只不过权重 W 对合法配置 = 1，非法配置 = 0。

3.3 核心张量 B 的 9 个指标

代码中定义的张量 B 有 9 个指标，每个维度为 2：

$$B(u, ul, l, ld, d, dr, r, ru, p)$$



每个指标的含义（维度都是 2）

- u, d, l, r : 上、下、左、右方向的列/行攻击信号
- ul, dr : 左上 $\rightarrow$ 右下对角线的攻击信号
- ur, ld : 右上 $\rightarrow$ 左下对角线的攻击信号（代码中 ru 和 ld ）
- p : 该格的占据状态（1= 空，2= 有皇后）

每个指标取值的意义：

- 值 = 1: 该方向没有攻击信号经过
- 值 = 2: 该方向有攻击信号经过（该方向已经有一个皇后在攻击）

3.4 B 张量非零元素的完整解读

B 总共 $2^9 = 512$ 个元素，其中只有 16 个非零 (= 1)，其余都是 0。每个非零元素对应一种合法的局部状态。

3.4.1 情况一：格子为空 ($p = 1$)，无皇后经过

$$B(1, 1, 1, 1, 1, 1, 1, 1, 1) = 1$$

意思：所有方向都没有攻击信号，这个格子是空的。完全合法。

3.4.2 情况二：格子为空 ($p = 1$)，有信号”透传”

当格子为空时，对角线上的攻击信号可以穿过这个格子。但规则是：

- 对角线信号必须成对出现：从一边进，从另一边出
- 例如：左上有信号进来 ($ul = 2$)，就必须从右下出去 ($dr = 2$)
- 列/行信号同理：上面有信号 ($u = 2$)，就必须向下传 ($d = 2$)

透传的例子

$B(2, 1, 1, 1, 2, 1, 1, 1, 1) = 1$ 列攻击信号：上 $\rightarrow$ 下透传

意思：上方有皇后在这一列攻击 ($u = 2$)，信号向下传递 ($d = 2$)，这个空格子只是把信号传下去。

$B(1, 2, 1, 1, 1, 2, 1, 1, 1) = 1$ 对角线信号：左上 $\rightarrow$ 右下透传

意思：左上方对角线有皇后 ($ul = 2$)，信号沿对角线向右下传 ($dr = 2$)。

还有多条信号同时透传的情况：

$B(2, 2, 1, 1, 2, 2, 1, 1, 1) = 1$ 列 + 对角线同时透传

$B(2, 2, 2, 1, 2, 2, 2, 1, 1) = 1$ 列 + 对角线 + 反对角线同时透传 (3 条信号)

透传规则总结 ($p = 1$ 时)：

入方向	出方向	含义
$u = 2$	$d = 2$	列攻击
$ul = 2$	$dr = 2$	$\searrow$ 对角线
$l = 2$	$r = 2$	行攻击
$ld = 2$	$ru = 2$	$\nearrow$ 反对角线

这 4 条通道相互独立，每条可以开 (2) 或关 (1)，所以空格子有 $2^4 = 16$ 种可能。但其中 $p = 2$ (有皇后) 的情况要另算，实际 $p = 1$ 的合法状态有 $2^4 = 16$ 种——不过等等，当 $p = 1$ 时还有一个 $B(1, 1, 1, 1, 2, 2, 2, 2, 2)$ 是 $p = 2$ 的情况。

让我重新梳理： $p = 1$ (空格子) 的非零元素是上面列出的 $2^4 - 1 = 15$ 个透传组合 (选择哪些通道开)，一共 $2^4 = 16$ 种 (包括全关)。但代码中 B 的最后一个指标是 p ，我们看到：

$p = 1$ 时有 15 个非零元素 (0 到 4 条信号透传的各种组合)。

3.4.3 情况三：格子有皇后 ($p = 2$)

$$B(1, 1, 1, 1, 2, 2, 2, 2, 2) = 1$$

意思：这里放了一个皇后！它没有从任何方向接收攻击信号 ($u = 1, ul = 1, l = 1, ld = 1$)，但向所有方向发出攻击信号 ($d = 2, dr = 2, r = 2, ru = 2$)。

为什么入方向都是 1？

如果 $u = 2$ （上方同列已有皇后）同时 $p = 2$ （这里也有皇后），就违反了列约束！所以：

$$B(2, \cdot, \cdot, \cdot, \cdot, \cdot, \cdot, \cdot, 2) = 0 \quad (\text{只要 } u = 2 \text{ 且 } p = 2)$$

类似地，如果任何入方向有攻击信号且这里有皇后， $B = 0$ 。

唯一合法的有皇后状态：所有入方向都无信号 ($=1$)，所有出方向都有信号 ($=2$)。

3.4.4 为什么只有 16 个非零元素

16 个非零元素 = 15（空格子透传）+ 1（放皇后）

1. 空格子 ($p = 1$)：4 条独立通道，每条开/关 $\rightarrow 2^4 = 16$ 种（但 $B(2, 2, 2, 2, 2, 2, 2, 2, 1) = 1$ 是全开的情况，代码中确实列出了）

实际上代码中 $p = 1$ 的非零元素：

- 0 条通道开：1 个（全 1）
- 1 条通道开： $\binom{4}{1} = 4$ 个
- 2 条通道开： $\binom{4}{2} = 6$ 个
- 3 条通道开： $\binom{4}{3} = 4$ 个
- 4 条通道开：0 个？

等等，让我重新数。看代码： $B(2, 2, 2, 2, 2, 2, 2, 2, 1) = 1$ ，这是 4 条全开且 $p = 1$ （空格子），意思是这个空格子上所有四个方向都有攻击线经过——完全合法，只是不能放皇后。

所以 $p = 1$ 有 $1 + 4 + 6 + 4 + 1 = 16$ 个。但这加上 $p = 2$ 的 1 个是 17 个？

让我重数代码... 代码中明确列出的是：

- 行 4： $B(1, 1, 1, 1, 1, 1, 1, 1, 1) = 1$
- 行 5： $B(2, 2, 2, 2, 2, 2, 2, 2, 1) = 1$ （4 条全开， $p = 1$ ）
- 行 7-10：4 个单通道
- 行 12-17：6 个双通道

- 行 19-22: 4 个三通道
- 行 24: $B(1, 1, 1, 1, 2, 2, 2, 2, 2) = 1$ ($p = 2$, 放皇后)

总计 $1 + 1 + 4 + 6 + 4 + 1 = 17$? 不对, $1 + 4 + 6 + 4 + 1 = 16$ 个 $p = 1$, 加上 1 个 $p = 2 = 17$ 个。

但标题说 16 个... 让我再数一下代码中 B 的非零元素: 第 4, 5, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 24 行, 共 17 行赋值。

总结: B 张量共 17 个非零元素 = 16 (空格子的各种透传组合) + 1 (放皇后)。

3.5 一个极简例子: 2×2 的情况

2×2 棋盘, 理解信号传播

2×2 棋盘只有 4 个格子。N=2 皇后问题要在 2×2 棋盘上放 2 个皇后使互不攻击——显然无解 ($Z_2 = 0$)。

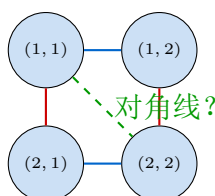
用张量网络来验证: 如果在 (1, 1) 放皇后, 它会向右发出行攻击信号 (到 (1, 2)), 向下发出列攻击信号 (到 (2, 1)), 向右下发出对角线信号 (到 (2, 2))。这三个格子全部被封锁, 无法放第二个皇后。其他位置同理。

4 从 9 阶张量到 2D 网络张量: 辅助张量的作用

4.1 问题: 对角线指标如何处理?

原始的 B 张量有 9 个指标 (8 个方向 + 1 个占据状态)。在正方格点上:

- 上下左右 4 个方向的连接很自然 (连到邻居格点)
- 但对角线方向呢? (i, j) 的右下邻居是 $(i + 1, j + 1)$, 不是直接的上下左右邻居



对角线连接会让网络变成非平面的, 无法用简单的 MPS/MPO 方法处理。

4.2 解决方案概述: 用 UE、DE、V0 把对角线变成上下左右

代码的巧妙之处是引入了三个辅助张量:

- **UE**: 把对角线信号编码 (合并) 进垂直和水平键

- **DE**: 把合并后的键解码（拆分）回独立的对角线信号
- **V0**: 对占据状态 p 求和（追踪所有配置）

其目标是：消除所有对角线连接，使张量网络变成纯粹的正方格子（只有上下左右的键），从而可以用 MPS/MPO 方法处理。

4.3 UE 张量详解：对角线信号的编码器

4.3.1 第一步：4 维 SWAP 张量

UE 最初定义为一个 4 阶张量，每个指标维度为 2：

$$\text{UE}(a, b, c, d) = \delta_{a,d} \cdot \delta_{b,c}$$

写出全部 $2^4 = 16$ 个元素，只有 4 个非零：

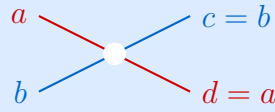
a	b	c	d	UE 值	含义
1	1	1	1	1	$\delta_{1,1}\delta_{1,1}$: 两对都匹配
1	2	2	1	1	$\delta_{1,1}\delta_{2,2}$: $a=d=1, b=c=2$
2	1	1	2	1	$\delta_{2,2}\delta_{1,1}$: $a=d=2, b=c=1$
2	2	2	2	1	$\delta_{2,2}\delta_{2,2}$: 两对都匹配

UE 的本质：SWAP 门

UE 做的事情是：

$$\text{输入}(a, b) \rightarrow \text{输出}(c, d) = (b, a)$$

它交换两个指标。 $\delta_{a,d}$ 保证 $d = a$ （第一个输入 = 第二个输出）， $\delta_{b,c}$ 保证 $c = b$ （第二个输入 = 第一个输出）。



SWAP: 两条线交叉

4.3.2 第二步：Reshape 为 $[2, 2, 4]$

代码将 UE 的后两个指标 (c, d) 合并为一个维度-4 的指标 k ：

$$\text{UE}_r(a, b, k) \quad \text{其中} \quad k = c + 2(d - 1)$$

这是 MATLAB 的列优先（column-major）排列。映射表：

c	d	合并后 k	物理含义
1	1	1	两条对角线都无信号
2	1	2	c 方向有信号, d 方向无
1	2	3	c 方向无信号, d 方向有
2	2	4	两条对角线都有信号

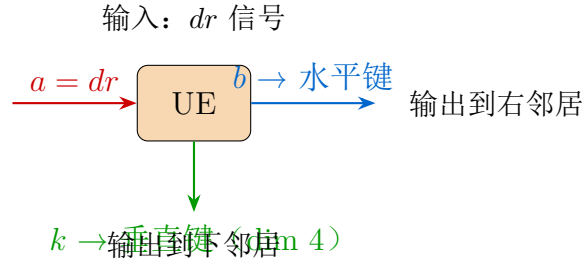
Reshape 后的 4 个非零元素:

a	b	k	UE_r	来源
1	1	1	1	$(c=1, d=1) \rightarrow k=1$
1	2	2	1	$(c=2, d=1) \rightarrow k=2$
2	1	3	1	$(c=1, d=2) \rightarrow k=3$
2	2	4	1	$(c=2, d=2) \rightarrow k=4$

4.3.3 UE 的作用方式

在 ncon 缩并中, UE 的三个指标分别连接到:

- 指标 a (dim 2): 与原始张量的 dr (右下对角线) 缩并
- 指标 b (dim 2): 输出到**水平键** (向右传递)
- 指标 k (dim 4): 输出到**垂直键** (向下传递)



具体例子: 当 $dr = 1$ (无对角线信号) 时

$UE_r(1, b, k)$ 的非零元素:

- $(b=1, k=1)$: 水平键输出 1 (无信号), 垂直键输出 1 (无信号)
- $(b=2, k=2)$: 水平键输出 2 (有信号), 垂直键输出 2

为什么 $dr = 1$ 却产生了两种输出?

因为 UE 的 SWAP 结构要求 $d = a = dr$ 且 $c = b$ 。当 $dr = 1$ 时, d 被固定为 1, 但 b (和 $c = b$) 是自由的。这意味着垂直键的 k 中, d 分量被锁定为 1, 但 c 分量可以取任何值——它携带的是从其他位置传来的对角线信息, 与当前格点的 dr 无关。

具体地:

- $(b=1, k=1)$: 对应 $(c=1, d=1) \rightarrow$ 既无来自别处的信号, 也无本格 dr 信号
- $(b=2, k=2)$: 对应 $(c=2, d=1) \rightarrow$ 有来自别处的信号在传递, 但本格 dr 无信号

具体例子: 当 $dr = 2$ (有对角线信号, 这里有皇后!) 时

$UE_r(2, b, k)$ 的非零元素:

- $(b=1, k=3)$: 水平键 1, 垂直键 3。对应 $(c=1, d=2)$: dr 信号存入垂直键的 d 分量
- $(b=2, k=4)$: 水平键 2, 垂直键 4。对应 $(c=2, d=2)$: dr 信号 + 其他信号都有

UE 的关键特性: 制造关联

UE 使得水平键的值 b 和垂直键的值 k 不再独立——它们通过 $c = b$ 的约束被关联起来。

这正是对角线约束被编码的方式: 对角线信号需要同时出现在水平和垂直方向, UE 保证了这种一致性。

如果水平键传出了 $b = 2$ (有对角线信号), 那么垂直键中 c 分量也必须是 2——这意味着斜下方的邻居也能“看到”这个信号。

4.4 DE 张量详解: 对角线信号的解码器

4.4.1 从单位矩阵出发

$$DE = \text{reshape}(I_4, [4, 2, 2])$$

I_4 是 4×4 单位矩阵。Reshape 后变成 3 阶张量, 三个指标维度分别为 $(4, 2, 2)$ 。

I_4 的非零元素在对角线上: $(1, 1), (2, 2), (3, 3), (4, 4)$ 。

MATLAB 列优先 reshape 的映射: 4×4 矩阵的元素 (row, col) 对应线性索引 $\text{row} + 4(\text{col} - 1)$, 而 $[4, 2, 2]$ 张量的元素 (k, i, j) 对应线性索引 $k + 4(i - 1) + 8(j - 1)$ 。

逐个计算 I_4 对角线元素的映射:

I_4 位置	线性索引	k	i	j	含义
$(1, 1)$	1	1	1	1	$k=1 \rightarrow$ 双无, 拆分为 $i=1, j=1$
$(2, 2)$	6	2	2	1	$k=2 \rightarrow$ 拆分为 $i=2, j=1$
$(3, 3)$	11	3	1	2	$k=3 \rightarrow$ 拆分为 $i=1, j=2$
$(4, 4)$	16	4	2	2	$k=4 \rightarrow$ 双有, 拆分为 $i=2, j=2$

验证线性索引

以 $I_4(2, 2)$ 为例：线性索引 $= 2 + 4 \times (2 - 1) = 6$ 。

在 $[4, 2, 2]$ 张量中： $k + 4(i - 1) + 8(j - 1) = 6$ ，解得 $k = 2, i = 2, j = 1$ 。✓

以 $I_4(3, 3)$ 为例：线性索引 $= 3 + 4 \times (3 - 1) = 11$ 。

在 $[4, 2, 2]$ 张量中： $11 - 8 = 3$ ，所以 $j = 2$ ，剩下 $k + 4(i - 1) = 3$ ，得 $k = 3, i = 1$ 。✓

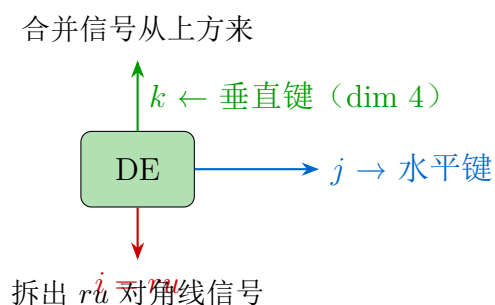
所以 DE 的 4 个非零元素是：

k (dim 4)	i (dim 2)	j (dim 2)	DE 值
1	1	1	1
2	2	1	1
3	1	2	1
4	2	2	1

4.4.2 DE 的作用方式

在 ncon 中，DE 的三个指标连接到：

- 指标 k (dim 4)：输入来自**垂直键**（从上方传来的合并信息）
- 指标 i (dim 2)：与原始张量的 ru （右上对角线）缩并
- 指标 j (dim 2)：输出到**水平键**（向右传递）



DE 的作用：拆分合并索引

DE 做的恰好是 UE 的逆操作：

UE 把 (c, d) 合并为 $k = c + 2(d - 1)$ ，同时将 $b = c$ 和 $a = d$ 传出。

DE 把 k 拆分回两个分量：

- $k = 1 \rightarrow (i=1, j=1)$ ：无信号
- $k = 2 \rightarrow (i=2, j=1)$ ： ru 方向有信号， j 方向无
- $k = 3 \rightarrow (i=1, j=2)$ ： ru 方向无信号， j 方向有信号（传给右邻居）
- $k = 4 \rightarrow (i=2, j=2)$ ：两者都有信号

直觉：UE 是编码器（合并），DE 是解码器（拆分）。

4.5 V0 张量：对占据状态求和

$$V_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \text{ 形状}[2, 1]$$

V0 与原始张量的 p 指标缩并：

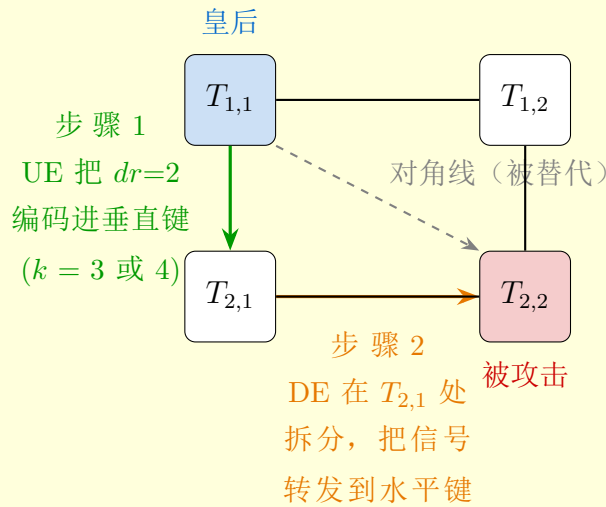
$$\sum_{p=1}^2 T(\dots, p) \cdot V_0(p) = T(\dots, 1) + T(\dots, 2)$$

效果：把”空格子”和”有皇后”两种状态加起来。这正是配分函数的求和——我们要对所有可能的占据配置求和，让合法的贡献 1，非法的贡献 0。

4.6 UE 与 DE 的协作：对角线信号的完整路由

完整路由例子：(1, 1) 的皇后攻击 (2, 2)

假设 (1, 1) 放了皇后， $dr = 2$ 。对角线信号如何到达 (2, 2)？



详细步骤：

1. 在 $T_{1,1}$ ：皇后使 $dr = 2$ 。UE 将 $a=dr=2$ 编码，产生 $k = 3$ （当 $b=1$ 时）或 $k = 4$ （当 $b=2$ 时）。 k 通过垂直键传给 $T_{2,1}$ 。
2. 在 $T_{2,1}$ ：DE 接收到 $k = 3$ ，拆分为 $(i=1, j=2)$ 。 $j=2$ 通过水平键传给 $T_{2,2}$ 。 $T_{2,2}$ 收到 $j=2$ 表示”有对角线攻击经过”，如果 $T_{2,2}$ 也想放皇后，就会因为入方向有信号而被 B 张量的零元素封锁。

4.7 Reshape 后的键维度解读

经过与 UE、DE、V0 缩并后，每个张量只剩 2-4 个指标，维度为 2、4 或 8：

键维度的物理含义

维度	编码内容	举例
2	1 条攻击线	角落处单条对角线
4	2 条攻击线	边缘处两条对角线 (2×2)
8	3 条攻击线	内部的列 + 两条对角线 ($2 \times 2 \times 2$)

为什么内部垂直键维度是 8

看内部张量 B 的垂直键：它需要传递 3 种信号：

1. **列攻击**：这一列上方/下方是否有皇后 $\rightarrow 1$ bit（维度 2）
2. **$\searrow$ 对角线**：经过这个键的下行对角线是否有皇后 $\rightarrow 1$ bit（维度 2）
3. **$\nearrow$ 对角线**：经过这个键的上行对角线是否有皇后 $\rightarrow 1$ bit（维度 2）

三者独立 $\rightarrow 2 \times 2 \times 2 = 8$ 。

具体地，垂直键的值 $\alpha \in \{1, 2, \dots, 8\}$ 编码为三元组 (c, d_1, d_2) ：

α	列攻击 c	$\searrow$ 对角线 d_1	$\nearrow$ 对角线 d_2
1	无	无	无
2	无	无	有
3	无	有	无
4	无	有	有
5	有	无	无
6	有	无	有
7	有	有	无
8	有	有	有

为什么左右水平键维度有的是 4 有的是 8

水平键需要传递的信号取决于它连接的是哪两列之间：

- **第 1 行的水平键**（维度 4）：第 1 行上方没有格子，所以不需要传列信号（因为列信号是垂直的）。只需传两条对角线信息 $\rightarrow 2 \times 2 = 4$
- **内部行的水平键**（维度 8）：需要传递行攻击 + 两条经过的对角线信息 $\rightarrow 2 \times 2 \times 2 = 8$

- 右端边界键（维度 2 或 4）：右边缘对角线更少

5 完整数值推导： $T_{1,1}$ 的每一个元素

下面以左上角 $T_{1,1}$ 为例，从头到尾追踪每一步缩并的数值，最终得到完整的 8×4 矩阵。

5.1 第一步：原始 9 阶张量（2 个非零元素）

$T_{1,1}$ 对应棋盘左上角（CLU），原始维度为：

$$T_{1,1}(u, ul, l, ld, d, dr, r, ru, p) \quad \text{形状}[1, 1, 1, 1, 2, 2, 2, 1, 2]$$

边界效应： u, ul, l, ld, ru 的维度都是 1（上方和左方没有邻居），所以这些指标被锁定为 1，只有 d, dr, r, p 是自由的。

只有 2 个非零元素：

u	ul	l	ld	d	dr	r	ru	p	值	物理含义
1	1	1	1	1	1	1	1	1	1	空格子，所有方向无信号
1	1	1	1	2	2	2	1	2	1	放了皇后，向 d, dr, r 发信号

为什么只有这 2 个？

- 这是左上角，上方和左方没有邻居 → 不可能有攻击信号从上方或左方进来
- 所以入方向 u, ul, l, ld 全是 1
- 要么空（全 1），要么放皇后（ $p=2$ ，向下 $d=2$ 、右下 $dr=2$ 、右 $r=2$ 发信号）
- $ru = 1$ 因为右上方没有邻居（这是第 1 行）

5.2 第二步：与 UE 缩并（ dr 信号编码）

ncon 指令：

`ncon({T{1,1}}, UE, V0), {[-1,-2,-3,-4,-5,1,-8,-7,2], [1,-9,-6], [2,-10]}`

各指标的映射：

张量	原始指标	ncon 标签	维度
$9^*T_{1,1}$	u	-1	1
	ul	-2	1
	l	-3	1
	ld	-4	1
	d	-5	2
	dr	1 (与 UE 缩并)	2
	r	-8	2
	ru	-7	1
	p	2 (与 V0 缩并)	2
3^*UE_r	指标 1 (a)	1 (与 dr 缩并)	2
	指标 2 (b)	-9	2
	指标 3 (k)	-6	4
2^*V0	指标 1	2 (与 p 缩并)	2
	指标 2	-10	1

缩并公式：

$$\text{tmp}(\underbrace{-1, -2, -3, -4}_{\text{全}=1}, \underbrace{-5}_d, \underbrace{-6}_k, \underbrace{-7}_{=1}, \underbrace{-8}_r, \underbrace{-9}_b, \underbrace{-10}_{=1}) = \sum_{a,p} T_{1,1}(\dots, d, a, r, \dots, p) \cdot UE_r(a, b, k) \cdot V_0(p)$$

由于 $u = ul = l = ld = ru = 1$ 且 $V_0(1) = V_0(2) = 1$ ，实际有效的求和是：

$$\text{tmp}(d, k, r, b) = \sum_{a=1}^2 T^{\text{eff}}(d, a, r) \cdot UE_r(a, b, k)$$

其中 T^{eff} 是原始张量对 p 求和后的有效张量（因为 V_0 把两个 p 值都加了起来，而这里每个 p 值对应不同的 (d, dr, r) ，所以效果等价于直接使用）。

5.2.1 逐项计算

从非零元素 **1**：($d=1, dr=1, r=1, p=1$)

$a = dr = 1$ ， $V_0(p=1) = 1$ 。查 $UE_r(1, b, k)$ 的非零值：

b	k	结果
1	1	$\text{tmp}(d=1, k=1, r=1, b=1) = 1 \times 1 \times 1 = \mathbf{1}$
2	2	$\text{tmp}(d=1, k=2, r=1, b=2) = 1 \times 1 \times 1 = \mathbf{1}$

从非零元素 **2**：($d=2, dr=2, r=2, p=2$)

$a = dr = 2$ ， $V_0(p=2) = 1$ 。查 $UE_r(2, b, k)$ 的非零值：

b	k	结果
1	3	$\text{tmp}(d=2, k=3, r=2, b=1) = 1 \times 1 \times 1 = \mathbf{1}$
2	4	$\text{tmp}(d=2, k=4, r=2, b=2) = 1 \times 1 \times 1 = \mathbf{1}$

所以中间结果 tmp 有 **4 个非零元素**（从 2 个变成 4 个，因为 UE 的 SWAP 结构为每个原始元素产生了 2 个输出）。

5.3 第三步：Reshape 为 $[8, 4]$ 矩阵

tmp 的完整维度是 $(1, 1, 1, 1, 2, 4, 1, 2, 2, 1)$ ，省略所有维度-1 的指标后，有效维度是 $(d:2, k:4, r:2, b:2)$ 。

Reshape 为 $[8, 4]$ ：

- 行指标 I (1-8)：合并 $(-1, -2, -3, -4, -5, -6) = (u, ul, l, ld, d, k)$
- 列指标 J (1-4)：合并 $(-7, -8, -9, -10) = (ru, r, b, V_0)$

由于 $u = ul = l = ld = ru = 1$ 且 V_0 维度为 1，有效的编码公式为：

$$I = 1 + (d - 1) + 2(k - 1)$$

$$J = 1 + (r - 1) + 2(b - 1)$$

编码实例

- $(d=1, k=1) \rightarrow I = 1 + 0 + 0 = 1$
- $(d=1, k=2) \rightarrow I = 1 + 0 + 2 = 3$
- $(d=2, k=3) \rightarrow I = 1 + 1 + 4 = 6$
- $(d=2, k=4) \rightarrow I = 1 + 1 + 6 = 8$
- $(r=1, b=1) \rightarrow J = 1 + 0 + 0 = 1$
- $(r=2, b=1) \rightarrow J = 1 + 1 + 0 = 2$
- $(r=1, b=2) \rightarrow J = 1 + 0 + 2 = 3$
- $(r=2, b=2) \rightarrow J = 1 + 1 + 2 = 4$

5.4 $T_{1,1}$ 的完整 8×4 矩阵

将 4 个非零元素映射到 (I, J) ：

d	k	r	b	I	J	值	物理来源
1	1	1	1	1	1	1	空格子，通道 1
1	2	1	2	3	3	1	空格子，通道 2
2	3	2	1	6	2	1	皇后，通道 1
2	4	2	2	8	4	1	皇后，通道 2

$T_{1,1}$ 的完整数值矩阵

$$T_{1,1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}_{8 \times 4}$$

32 个元素中只有 4 个非零，全部等于 1。

5.5 行指标和列指标的完整含义

5.5.1 行指标 I （垂直键，维度 8）= 向下传递的信息

I	d	k	列攻击	对角线 (c, d_{diag})	含义
1	1	1	无	(1,1)= 双无	下方完全安全
2	2	1	有	(1,1)= 双无	列有皇后攻击，对角线无
3	1	2	无	(2,1)= c 有	对角线路由通道 c 有信号
4	2	2	有	(2,1)= c 有	列攻击 + 对角线 c 通道
5	1	3	无	(1,2)= d 有	dr 对角线有信号
6	2	3	有	(1,2)= d 有	列攻击 + dr 对角线
7	1	4	无	(2,2)= 双有	两条对角线都有信号
8	2	4	有	(2,2)= 双有	列攻击 + 两条对角线

其中 k 编码 (c, d_{diag}): c 是路由通道（从 UE 的 b 复制来）， d_{diag} 是 dr 的实际信号（从 UE 的 $\delta_{a,d}$ 传来）。

5.5.2 列指标 J （水平键，维度 4）= 向右传递的信息

J	r	b	含义
1	1	1	无行攻击，路由通道无信号
2	2	1	行攻击，路由通道无信号
3	1	2	无行攻击，路由通道有信号
4	2	2	行攻击 + 路由通道有信号

5.6 4 个非零元素的物理解读

逐个解读

(1) $T_{1,1}(1,1) = 1$ ：空格子，安静通道

- 行指标 $I=1$ ：下方无列攻击，无对角线信号 $\rightarrow$ 下方格子完全自由
- 列指标 $J=1$ ：右方无行攻击，路由通道空 $\rightarrow$ 右方格子完全自由
- 这是“(1,1) 为空，且不传递任何信号”的状态

(2) $T_{1,1}(3,3) = 1$ ：空格子，路由通道激活

- $I=3$ ：下方无列攻击，但对角线路由通道 $c=2$ （有信号）
- $J=3$ ：右方无行攻击，但路由通道 $b=2$ （有信号）
- b 和 c 通过 UE 的 $\delta_{b,c}$ 约束绑定 ($b=c=2$)
- 物理含义：(1,1) 为空，但键上的路由通道是”激活”的。这是 UE SWAP 结构产生的——它允许对角线信息从其他位置经过这个键传递。

为什么空格子会有”激活”的路由通道？

这看起来不合理——左上角不会有对角线信号经过。

答案是：这个状态在全局缩并中贡献为零。 $T_{1,1}$ 是局部张量，它列出了所有局部合法的状态。但当与邻居张量缩并后， $I=3, J=3$ 这个通道找不到匹配的邻居状态（因为没有信号源），所以它对最终结果 Z_N 的贡献是 0。

张量网络的威力在于：全局约束通过局部张量的缩并自动涌现。

(3) $T_{1,1}(6,2) = 1$ ：皇后，通道 1

- $I=6$ ： $d=2$ （列攻击向下）， $k=3$ 即 ($c=1, d_{\text{diag}}=2$)： dr 对角线有信号
- $J=2$ ： $r=2$ （行攻击向右）， $b=1$ （路由通道空）

- 路由通道 $b=1$ 和 $c=1$ 匹配 ($\delta_{b,c}$)
- 含义: 皇后在 $(1,1)$, 向下发列攻击, 向右发行攻击, dr 对角线信号编码在垂直键中

(4) $T_{1,1}(8,4) = 1$: 皇后, 通道 2

- $I=8$: $d=2$ (列攻击), $k=4$ 即 ($c=2, d_{\text{diag}}=2$): dr 信号 + 路由通道 c 都激活
- $J=4$: $r=2$ (行攻击), $b=2$ (路由通道激活)
- 同样 $b=c=2$ 满足 $\delta_{b,c}$
- 含义: 与 (3) 相同的物理配置 (皇后在 $(1,1)$), 但路由通道处于不同状态

为什么一个皇后配置产生了 2 个非零元素 ((3) 和 (4))?

因为 UE 的 SWAP 结构在 b 维度上有 2 个自由度。这两个元素编码了同一个物理事实 (皇后在 $(1,1)$), 但在路由通道的不同状态上。

在全局缩并中, 具体哪个路由状态会”存活”取决于邻居张量的状态——网络自动选择出一致的全局路由方案。

5.7 验证: 用 MATLAB 计算

如果你有 MATLAB, 可以运行以下代码验证:

```
addpath(genpath("./ncon"));
[Td] = GetNqueensTensors(4);
Td{1,1}           % 应输出 8x4 矩阵
find(Td{1,1})      % 应返回 [1; 19; 14; 32] 即 (1,1), (3,3), (6,2), (8,4)
```

(注意 MATLAB 的 `find` 返回列优先线性索引: $1 = (1,1)$, $19 = 6 + 8 \times (3 - 1) = (6,3)$... 不对, 8×4 矩阵列优先索引 (I,J) 对应 $I + 8(J - 1)$ 。所以 $(6,2) \rightarrow 6 + 8 = 14$, $(3,3) \rightarrow 3 + 16 = 19$, $(8,4) \rightarrow 8 + 24 = 32$ 。✓)

6 MPS：矩阵乘积态

6.1 什么是 MPS

MPS 的定义

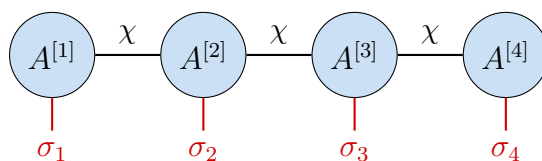
MPS (Matrix Product State) 是一维张量链： L 个张量从左到右排列，相邻张量通过**虚拟键**（virtual bond）连接。

对于一个 L 个格点的 MPS：

$$|\Psi\rangle = \sum_{\sigma_1, \dots, \sigma_L} A_{\sigma_1}^{[1]} A_{\sigma_2}^{[2]} \cdots A_{\sigma_L}^{[L]} |\sigma_1 \sigma_2 \cdots \sigma_L\rangle$$

其中：

- $A_{\sigma_j}^{[j]}$ 是一个**矩阵**（对于边界是向量）
- σ_j 是**物理指标**（朝上或朝下的腿）
- 矩阵乘法自动完成了虚拟键的缩并



χ = 虚拟键维度（bond dimension）， σ_j = 物理指标

具体例子：4 个格点的 MPS

假设物理维度 $d = 2$ （每个格点两种状态），虚拟键维度 $\chi = 3$ ：

- $A^{[1]}$ ：形状 $[d, \chi] = [2, 3]$ （左端点，没有左键）
- $A^{[2]}$ ：形状 $[\chi, d, \chi] = [3, 2, 3]$
- $A^{[3]}$ ：形状 $[\chi, d, \chi] = [3, 2, 3]$
- $A^{[4]}$ ：形状 $[\chi, d] = [3, 2]$ （右端点，没有右键）

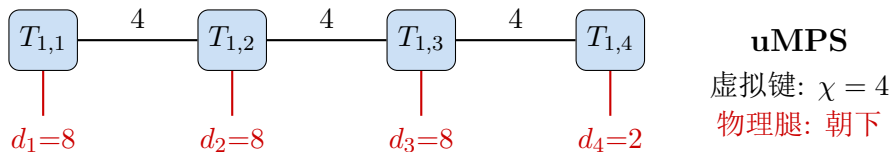
对于某个特定的物理配置 $(\sigma_1, \sigma_2, \sigma_3, \sigma_4)$ ，振幅为：

$$\Psi_{\sigma_1 \sigma_2 \sigma_3 \sigma_4} = \sum_{\alpha, \beta, \gamma} A_{\sigma_1, \alpha}^{[1]} A_{\alpha, \sigma_2, \beta}^{[2]} A_{\beta, \sigma_3, \gamma}^{[3]} A_{\gamma, \sigma_4}^{[4]}$$

这就是一连串矩阵乘法！

6.2 第 1 行作为 uMPS

在 N 皇后问题中，第 1 行的 4 个张量构成一个 MPS：

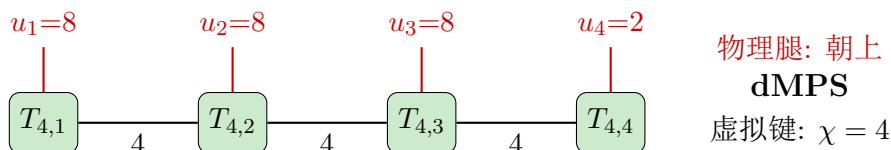


张量形状对应关系：

格点	形状	指标含义
$T_{1,1}$	$[8, 4]$	(物理腿 $d:8$, 右键 $r:4$)
$T_{1,2}$	$[4, 8, 4]$	(左键 $l:4$, 物理腿 $d:8$, 右键 $r:4$)
$T_{1,3}$	$[4, 8, 4]$	(左键 $l:4$, 物理腿 $d:8$, 右键 $r:4$)
$T_{1,4}$	$[4, 2]$	(左键 $l:4$, 物理腿 $d:2$)

物理腿朝下——因为第 1 行要和第 2 行连接。

6.3 第 4 行作为 dMPS



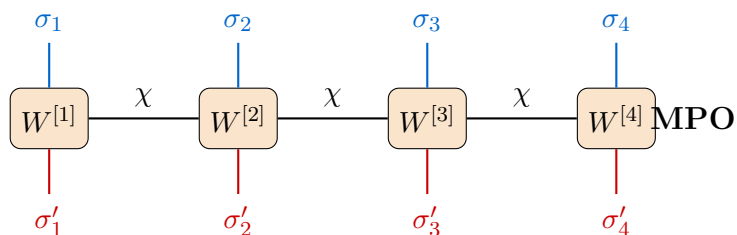
7 MPO：矩阵乘积算符

7.1 什么是 MPO

MPO 的定义

MPO (Matrix Product Operator) 是一维算符链。与 MPS 相比，每个张量多了一个指标——MPS 只有一个物理腿，MPO 有两个物理腿（上和下）。

$$\hat{O} = \sum_{\sigma, \sigma'} W_{\sigma_1 \sigma'_1}^{[1]} W_{\sigma_2 \sigma'_2}^{[2]} \cdots W_{\sigma_L \sigma'_L}^{[L]} |\sigma\rangle \langle \sigma'|$$

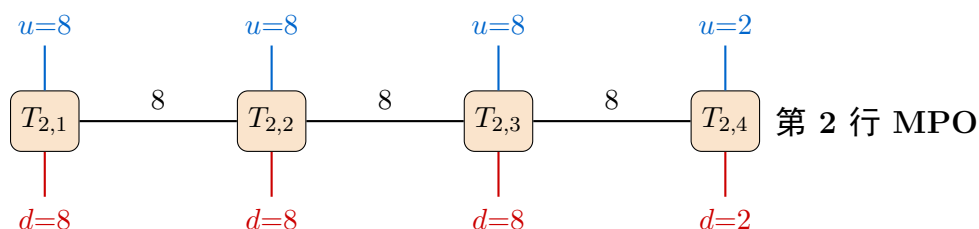


MPS vs MPO 的区别

	MPS	MPO
物理腿数	1 (朝上或朝下)	2 (上 + 下)
身份	"态" (向量)	"算符" (矩阵)
中间张量形状	$[\chi, d, \chi]$	$[\chi, d, d, \chi]$ (或 $[d, \chi, d, \chi]$)
类比	量子态 $ \Psi\rangle$	哈密顿量 $\hat{H}$ 或转移矩阵

7.2 第 2、3 行作为 MPO

在 4×4 N 皇后问题中，第 2 行和第 3 行各构成一个 MPO：



张量形状：

格点	形状	指标含义
$T_{2,1}$ (EL)	$[8, 8, 8]$	(上:8, 下:8, 右键:8)
$T_{2,2}$ (B)	$[8, 8, 8, 8]$	(上:8, 左键:8, 下:8, 右键:8)
$T_{2,3}$ (B)	$[8, 8, 8, 8]$	(上:8, 左键:8, 下:8, 右键:8)
$T_{2,4}$ (ER)	$[2, 8, 2]$	(上:2, 左键:8, 下:2)

注意左端和右端的不对称性

- 左端 $T_{2,1}$ ：没有左键（左边界），上下物理腿都是 8（要传完整的列 + 两对角线信息）
- 右端 $T_{2,4}$ ：没有右键（右边界），上下物理腿只有 2（右边缘只剩一条对角线信息）
- 这种不对称性来自于对角线在边界处的消失

8 核心操作：MPO 作用于 MPS

这是整个方法最关键的步骤。我们将逐个格点详细展示这个过程。

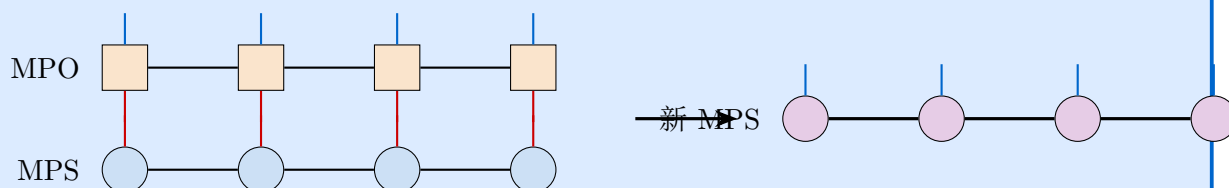
8.1 概览

MPO $\times$ MPS = 新的 MPS

将一个 MPO ”作用” 在一个 MPS 上，就像矩阵乘以向量：

$$|\Psi'\rangle = \hat{O}|\Psi\rangle$$

图形上：把 MPO 叠在 MPS 上方，将 MPO 的下腿与 MPS 的上腿缩并：



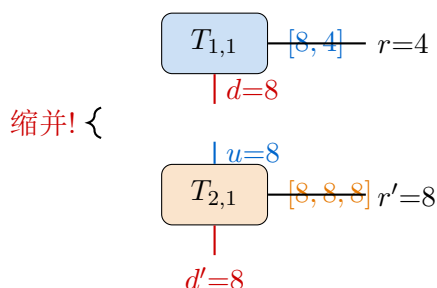
关键变化：新 MPS 的虚拟键维度 = MPO 虚拟键 $\times$ 旧 MPS 虚拟键。

这就是为什么键维度会指数增长！

8.2 逐格点详解：uMPS 吸收第 2 行 MPO

初始 uMPS 是第 1 行张量，MPO 是第 2 行张量。我们逐格点展示缩并过程。

8.2.1 格点 $j = 1$ （左端点）



操作：将 MPO 的上腿 u （维度 8）与 MPS 的下腿 d （维度 8）缩并。

$$\text{新 } M_{d', (r', r)}^{[1]} = \sum_{\alpha=1}^8 T_{2,1}(\alpha, d', r') \cdot T_{1,1}(\alpha, r)$$

- 缩并掉的指标：上-下连接（维度 8）
- 剩余指标： d' （新的下腿，维度 8）、 r' （MPO 的右键，维度 8）、 r （MPS 的右键，维度 4）
- 中间结果形状： $[8, 8, 4]$
- **Reshape:** 将 r' 和 r 合并为一个右键 $\rightarrow [8, 32]$

为什么右键变成了 32?

直觉：新的 MPS 需要同时记住两层信息——

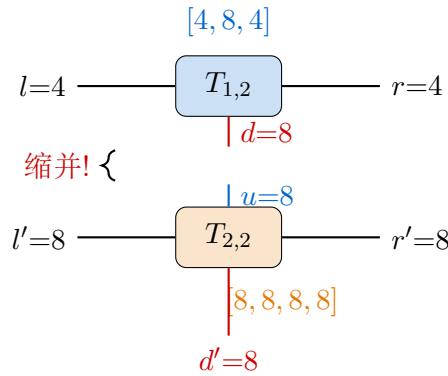
1. 第 1 行内部传递的信息（原虚拟键 $\chi = 4$ ）
2. 第 2 行内部传递的信息（MPO 虚拟键 $\chi' = 8$ ）

两者独立，所以用张量积 $4 \times 8 = 32$ 来编码。

具体地，新右键的 32 个状态 (α, β) 中：

- $\alpha \in \{1, \dots, 8\}$ ：第 2 行向右传递的攻击线状态
- $\beta \in \{1, \dots, 4\}$ ：第 1 行向右传递的对角线状态

8.2.2 格点 $j = 2$ （中间点）



操作：MPO 的上腿 u （8）与 MPS 的下腿 d （8）缩并。

$$\text{新 } M_{(l', l), d', (r', r)}^{[2]} = \sum_{\alpha=1}^8 T_{2,2}(\alpha, l', d', r') \cdot T_{1,2}(l, \alpha, r)$$

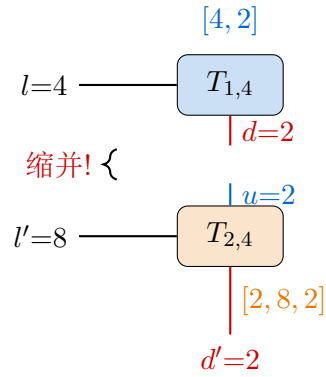
中间结果形状： $[8, 4, 8, 8, 4]$ （分别对应 l', l, d', r', r ）

Reshape：合并左键 $(l', l) \rightarrow 8 \times 4 = 32$ ，合并右键 $(r', r) \rightarrow 8 \times 4 = 32$

8.2.3 格点 $j = 3$ （同 $j = 2$ ）

完全相同的操作。新 uMPS $\{3\}$ ： $[32, 8, 32]$ 。

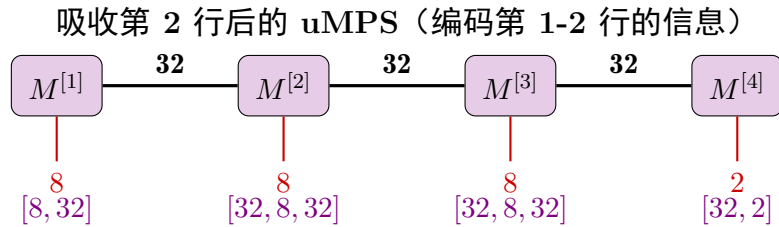
8.2.4 格点 $j = 4$ (右端点)



注意这里 $u = d = 2$ ，因为右边界只有一条对角线！

中间结果： $[8, 4, 2]$ (l', l, d') $\rightarrow$ Reshape 左键 $\rightarrow [32, 2]$

8.3 吸收后的 uMPS 全貌



键维度的指数增长

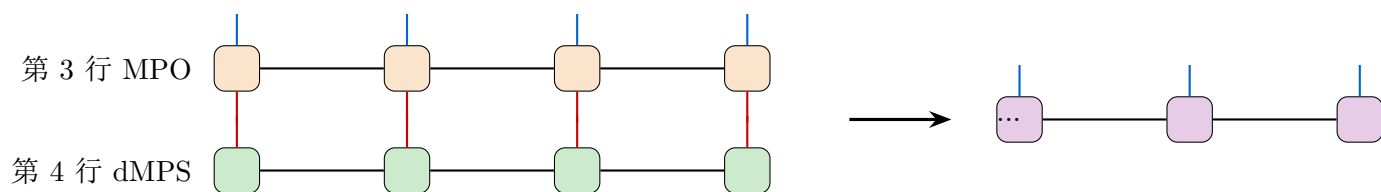
阶段	虚拟键维度 χ	编码了几行
初始 (第 1 行)	4	1 行
吸收第 2 行后	$4 \times 8 = 32$	2 行
若继续吸收第 3 行	$32 \times 8 = 256$	3 行
若继续...	$256 \times 8 = 2048$	4 行

每吸收一行， χ 乘以 MPO 的虚拟键维度。这就是为什么精确计算只能处理中等大小的 N —— χ 指数增长！对于近似计算，可以在每步之后用 SVD 截断到固定的 $\chi_{\max}$ 。

9 下方 MPS 吸收 MPO (反方向)

9.1 dMPS 吸收第 3 行

过程完全对称，只是方向相反：MPO 的下腿与 dMPS 的上腿缩并。



以 $j = 1$ 为例：

$$\text{新 } D_{u', (r', r)}^{[1]} = \sum_{\alpha=1}^8 T_{3,1}(u', \alpha, r') \cdot T_{4,1}(\alpha, r)$$

这里 $T_{3,1}$ 的下腿 $d = \alpha$ 与 $T_{4,1}$ 的上腿 $u = \alpha$ 缩并。

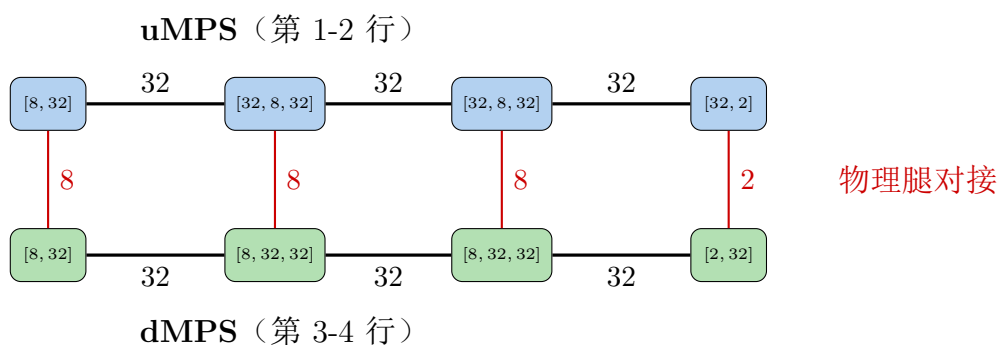
结果与上方完全对称：

格点	新 dMPS 形状	指标
$j = 1$	$[8, 32]$	(物理腿 $u:8$, 右键: 32)
$j = 2$	$[8, 32, 32]$	(物理腿 $u:8$, 左键: 32 , 右键: 32)
$j = 3$	$[8, 32, 32]$	(物理腿 $u:8$, 左键: 32 , 右键: 32)
$j = 4$	$[2, 32]$	(物理腿 $u:2$, 左键: 32)

10 最终缩并：两个 MPS 的内积

10.1 全局图景

现在我们有两个 MPS 在第 2-3 行之间对接：



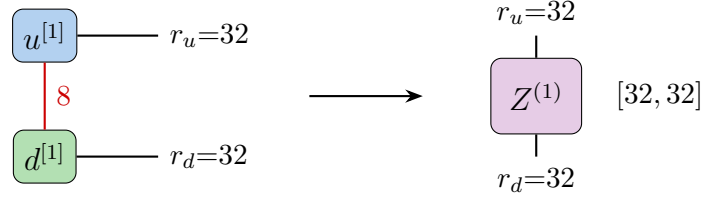
内积的计算 = 缩并所有红线（物理腿）和所有黑线（虚拟键），最终得到一个标量 Z_4 。

10.2 拉链式缩并

不能一次全缩并（太贵了）。采用从左到右逐列的方式：

10.2.1 第 1 列

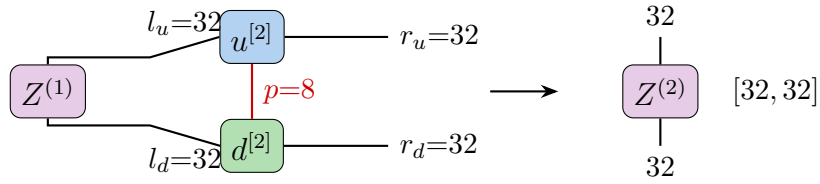
$$Z_{r_u, r_d}^{(1)} = \sum_{p_1=1}^8 u_{p_1, r_u}^{[1]} \cdot d_{p_1, r_d}^{[1]}$$



结果 $Z^{(1)}$ 是一个 32×32 的矩阵。

10.2.2 第 2 列

$$Z_{r_u, r_d}^{(2)} = \sum_{l_u, l_d, p} Z_{l_u, l_d}^{(1)} \cdot u_{l_u, p, r_u}^{[2]} \cdot d_{p, l_d, r_d}^{[2]}$$



关键：三个指标 l_u (32)、 l_d (32)、 p (8) 全部被求和掉，只留下 r_u (32) 和 r_d (32)。

计算量： $32 \times 32 \times 8 \times 32 \times 32 \approx 2.7 \times 10^7$ 次乘法。

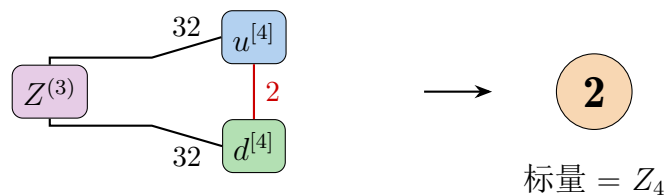
10.2.3 第 3 列：同第 2 列

$Z^{(3)}$ ：形状 $[32, 32]$ 。

10.2.4 第 4 列（最后一列）

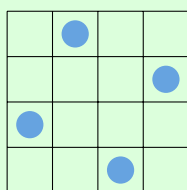
$$Z_4 = \sum_{l_u, l_d, p} Z_{l_u, l_d}^{(3)} \cdot u_{l_u, p}^{[4]} \cdot d_{p, l_d}^{[4]}$$

所有指标全部缩并 $\rightarrow$ 得到标量！

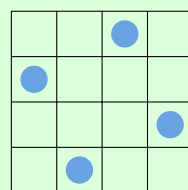


$Z_4 = 2$: 4 皇后有 2 个解!

这两个解分别是:

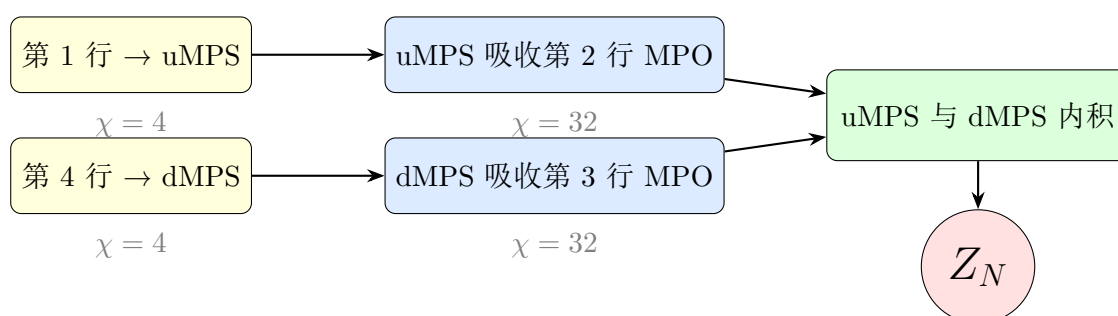


解 1



解 2

11 完整流程总结



为什么从两头往中间缩并?

如果只从上往下, 缩并 $N - 1$ 行 MPO 后 $\chi = 4 \times 8^{N-2}$ 。

从两头往中间, 各只缩并 $\sim N/2$ 行, 最大 $\chi \approx 4 \times 8^{N/2-1}$, 指数减半!

对于 $N = 8$:

- 单方向: $\chi_{\max} = 4 \times 8^6 = 1,048,576$
- 双方向: $\chi_{\max} = 4 \times 8^2 = 256$ (快了 4096 倍!)

11.1 计算复杂度

对于 $N \times N$ 的棋盘:

- MPO 虚拟键维度 $D = 8$
- 吸收 k 行后 MPS 虚拟键 $\chi_k \sim 4 \times 8^{k-1}$ (精确缩并, 不截断)
- 总计算量主导项: $O(N \cdot \chi_{\max}^2 \cdot D^2)$
- 最大 $\chi \approx 4 \times 8^{N/2-1}$, 所以复杂度 $\sim O(N \cdot 8^N)$

虽然仍是指数级, 但比暴力枚举 2^{N^2} 好得多 (暴力需要检查所有格子的所有占据方式)。

11.2 如何拓展到更大的 N ?

近似方法：SVD 截断

在每次 MPO 吸收后，对新 MPS 做 SVD 分解，只保留最大的 $\chi_{\max}$ 个奇异值：

$$M^{[j]} \approx U \cdot S \cdot V^\dagger \quad (\text{只保留前 } \chi_{\max} \text{ 个})$$

这样虚拟键维度被控制在 $\chi_{\max}$ ，但会引入近似误差。 $\chi_{\max}$ 越大越精确。

当前代码没有做截断（精确计算），所以只能算到 $N \sim 20$ 。

12 附录：所有张量的详细形状一览

12.1 4×4 所有格点张量

	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$i = 1$	$T_{1,1}: [8, 4]$ (CLU) (d, r)	$T_{1,2}: [4, 8, 4]$ (EU) (l, d, r)	$T_{1,3}: [4, 8, 4]$ (EU) (l, d, r)	$T_{1,4}: [4, 2]$ (CRU) (l, d)
$i = 2$	$T_{2,1}: [8, 8, 8]$ (EL) (u, d, r)	$T_{2,2}: [8, 8, 8, 8]$ (B) (u, l, d, r)	$T_{2,3}: [8, 8, 8, 8]$ (B) (u, l, d, r)	$T_{2,4}: [2, 8, 2]$ (ER) (u, l, d)
$i = 3$	$T_{3,1}: [8, 8, 8]$ (EL) (u, d, r)	$T_{3,2}: [8, 8, 8, 8]$ (B) (u, l, d, r)	$T_{3,3}: [8, 8, 8, 8]$ (B) (u, l, d, r)	$T_{3,4}: [2, 8, 2]$ (ER) (u, l, d)
$i = 4$	$T_{4,1}: [8, 4]$ (CLD) (u, r)	$T_{4,2}: [8, 4, 4]$ (ED) (u, l, r)	$T_{4,3}: [8, 4, 4]$ (ED) (u, l, r)	$T_{4,4}: [2, 4]$ (CRD) (u, l)

12.2 缩并过程中的 MPS 形状变化

阶段	$j = 1$	$j = 2$	$j = 3$	$j = 4$
初始 uMPS (第 1 行)	[8, 4]	[4, 8, 4]	[4, 8, 4]	[4, 2]
吸收第 2 行后	[8, 32]	[32 , 8, 32]	[32 , 8, 32]	[32 , 2]
初始 dMPS (第 4 行)	[8, 4]	[8, 4, 4]	[8, 4, 4]	[2, 4]
吸收第 3 行后	[8, 32]	[8, 32 , 32]	[8, 32 , 32]	[2, 32]
内积 $Z^{(j)}$	[32, 32]	[32, 32]	[32, 32]	2 (标量)

13 直觉总结

一句话理解整个方法

将 2D 棋盘按行切成 1D 条带：

1. 每一行是一个 MPS 或 MPO
2. 将 MPO 逐行”压”进 MPS (类似矩阵乘向量)
3. 最终两个 MPS 做内积 $\rightarrow$ 得到合法配置数

键维度 χ 的增长反映了跨行纠缠/关联的复杂度——需要记住多少关于上方/下方行的信息。

类比：为什么叫”矩阵乘积”？

回想 1D 系统的配分函数：

$$Z = \text{Tr}(T^N) = \text{Tr}(T \cdot T \cdot T \cdots T)$$

这就是一连串矩阵乘法 (转移矩阵法)。

MPS/MPO 方法是它的 **2D 推广**：

- 1D 的转移矩阵 $T \rightarrow$ 2D 的 MPO (矩阵变成了矩阵的”链”)
- 1D 的 $\text{Tr} \rightarrow$ 2D 的 MPS 内积
- 1D 的 $T^N \rightarrow$ 2D 的逐行 MPO 吸收

